学习报告

姓　　名：冯璨

日　　期：2026 年 3 月 17 日

1 线性回归

1.1 定义

具有线性特征的模型 $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$，通过随机梯度下降的方式，迭代参数 $\mathbf{w}$ 和 b 使得损失函数 $loss(\mathbf{w}, b)$ 最小。

1.2 代码实现

以下是基于 Pytorch 实现的简单线性回归示例代码：

```python
import numpy as np
import torch
from torch.utils import data
from d2l import torch as d2l

true_w = torch.tensor([2, -3.4])
true_b = 4.2
features, labels = d2l.synthetic_data(true_w, true_b, 1000)

def load_array(data_arrays, batch_size, is_train=True):
    dataset = data.TensorDataset(*data_arrays)
    return data.DataLoader(dataset, batch_size, shuffle=
        is_train)

batch_size = 10
data_iter = load_array((features, labels), batch_size)

from torch import nn
net = nn.Sequential(nn.Linear(2, 1))

net[0].weight.data.normal_(0, 0.01)
net[0].bias.data.fill_(0)

loss = nn.MSELoss()
```

```python

trainer = torch.optim.SGD(net.parameters(), lr=0.03)

num_epochs = 3
for epoch in range(num_epochs):
    for X, y in data_iter:
        l = loss(net(X) ,y)
        trainer.zero_grad()
        l.backward()
        trainer.step()
    l = loss(net(features), labels)
    print(f'epoch {epoch + 1}, loss {l:f}')

    w = net[0].weight.data
print('w的估计误差: ', true_w - w.reshape(true_w.shape))
b = net[0].bias.data
print('b的估计误差: ', true_b - b)
```

上述代码通过线性回归模型拟合了一组随机生成的线性数据，通过学习使得所求参数与正确参数结果基本一致。

2 softmax 回归

2.1 定义

对于一个分类问题，用线性模型对每一个类别计算出其对应的置信度 $\mathbf{o} = \mathbf{X}\mathbf{w} + b$, 再用 $\mathbf{o}$ 的指数概率计算 $\hat{y}$。其损失函数用交叉熵 $l(\mathbf{y}, \hat{\mathbf{y}}) = -\log(\hat{y}_y)$ 来表示。

2.2 代码实现

以下是基于 Pytorch 实现的 softmax 回归示例代码：

```python
import torch
```

```python
from torch import nn
from d2l import torch as d2l

batch_size = 256
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size)

net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))

def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

net.apply(init_weights);

loss = nn.CrossEntropyLoss(reduction='none')
trainer = torch.optim.SGD(net.parameters(), lr=0.1)
num_epochs = 10
d2l.train_ch6(net, train_iter, test_iter, num_epochs, 0.1, d2l.
    try_gpu())
```

3 多层感知机

3.1 定义

多层感知机是一种类 softmax 回归，可用于处理线性模型会出错的 XOR 问题。在 I/O 层之间设置一个隐藏层，具体地，输入层为 $\mathbf{x} \in R^n$，隐藏层参数为 $\mathbf{W_1} \in R^{m\times n}$，$b_1 \in R^m$，输出层参数为 $\mathbf{W_2} \in R^{m\times k}$，$\mathbf{b_2} \in R^k$。即

$$\mathbf{h} = \sigma(\mathbf{W_1}x + b_1)$$

$$\mathbf{o} = \mathbf{W_2^T}h + \mathbf{b_2}$$

$$\mathbf{y} = softmax(\mathbf{o})$$

其中 σ 是一个非线性的激活函数，用于实现 XOR 数据的转向。

3.2 代码实现

以下是基于 Pytorch 实现的多层感知机示例代码：

```
import torch
from torch import nn
from d2l import torch as d2l

net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    nn.Linear(256, 10))

def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

net.apply(init_weights);

batch_size, lr, num_epochs = 256, 0.1, 10
loss = nn.CrossEntropyLoss(reduction='none')
trainer = torch.optim.SGD(net.parameters(), lr=lr)

train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size)
d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr,d2l.
    try_gpu())
```

4 拟合优化

4.1 过拟合与欠拟合

训练误差是指，模型在训练数据集上计算得到的误差。泛化误差是指，模型应用在同样从原始样本的分布中抽取的无限多数据样本时，模型误差的期望。

将模型在训练数据上拟合的比在潜在分布中更接近的现象称为过拟合，反之则为欠拟合，用于对抗过拟合的技术称为正则化。

4.2 权重衰减

权重衰减指在损失函数中加入权重的 L2 正则项，惩罚过大的参数，迫使权重取值更小。其作用是抑制过拟合，让模型更简单、泛化能力更强。

4.3 丢弃法 Dropout

训练过程中，以设定概率随机将部分神经元输出置为 0，使其暂时不参与网络计算。其作用也是抑制过拟合，让模型不依赖少数神经元，更鲁棒。

5 深度学习计算

5.1 层与块

为了实现这些复杂的网络，我们引入了神经网络块的概念。块（block）可以描述单个层、由多个层组成的组件或整个模型本身。使用块进行抽象的一个好处是可以将一些块组合成更大的组件，这一过程通常是递归的

以下是基于 Pytorch 实现的自定义顺序层示例代码：

```python
import torch
from torch import nn
from torch.nn import functional as F

class MySequential(nn.Module):
    def __init__(self, *args):
        super().__init__()
        for idx, module in enumerate(args):
            # 这里，module是Module子类的一个实例。我们把它保存在'Module'类的成员
            # 变量_modules中。_module的类型是OrderedDict
            self._modules[str(idx)] = module
```

```python

    def forward(self, X):
        # OrderedDict保证了按照成员添加的顺序遍历它们
        for block in self._modules.values():
            X = block(X)
        return X
```

6 卷积神经网络

6.1 卷积层

对全连接层用平移不变性和局部性得到卷积层，例如对图片而言，将某个像素及其附近像素的加权和得到一个新的图片就是卷积过程。

以下是基于 Pytorch 实现的简单卷积层示例代码：

```python
import torch
from torch import nn
from d2l import torch as d2l
#卷积的实现
def corr2d(X, K):
    h, w = K.shape
    Y = torch.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = (X[i:i + h, j:j + w] * K).sum()
    return Y
class Conv2D(nn.Module):
    def __init__(self, kernel_size):
        super().__init__()
        self.weight = nn.Parameter(torch.rand(kernel_size))
        self.bias = nn.Parameter(torch.zeros(1))

    def forward(self, x):
```

```
        return corr2d(x, self.weight) + self.bias
```

6.2 填充和步幅

在连续多次应用卷积层会丢失较多边缘像素，因此采用填充的操作，通常采用的填充元素是 0。由于输出形状会变成 $(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1)$，因此取填充大小 $p_h = k_h - 1$，$p_w = k_w - 1$

步幅是指在计算卷积是，移动 kernel 的移动距离，提高步幅可以加速输出结果的缩小。

填充和步幅都是卷积层的超参数。

6.3 多输入多输出通道

当输入包含多个通道时，需要构造一个与输入数据具有相同输入通道数的卷积核，以便与输入数据进行卷积运算，最后将每个通道的值相加。

用 c_i 和 c_o 分别表示输入和输出通道的数目，并让 k_h 和 k_w 为卷积核的高度和宽度。为了获得多个通道的输出，我们可以为每个输出通道创建一个形状为 $c_i \times k_h \times k_w$ 的卷积核张量，这样卷积核的形状是 $c_o \times c_i \times k_h \times k_w$。在互相关运算中，每个输出通道先获取所有输入通道，再以对应该输出通道的卷积核计算出结果。

6.4 池化层

通常当我们处理图像时，我们希望逐渐降低隐藏表示的空间分辨率、聚集信息，这样随着我们在神经网络中层叠的上升，每个神经元对其敏感的感受野就越大。池化层的目的：降低卷积层对位置的敏感性，同时降低对空间降采样表示的敏感性。

池化层的池运算不包含参数，其运算是确定性的，通常计算池化窗口中所有元素的最大值或平均值。这些操作分别称为最大池化层和平均池化层。

6.5 经典卷积神经网络 LeNet

LeNet 由两个卷积层和三个全连接层组成,以下是基于 Pytorch 实现的 LeNet 模型学习手写数字示例代码:

```
import torch
from torch import nn
from d2l import torch as d2l

net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))

X = torch.rand(size=(1, 1, 28, 28), dtype=torch.float32)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__,'output shape: \t',X.shape)
batch_size = 256
transform = transforms.ToTensor()
train_dataset = datasets.MNIST(root="../data", train=True,
   download=True, transform=transform)
test_dataset = datasets.MNIST(root="../data", train=False,
   download=True, transform=transform)
features, labels = next(iter(train_iter))
plt.figure(figsize=(9,9))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(features[i].squeeze(), cmap="gray")
    plt.title(f"label:{labels[i]}")
```

```
    plt.axis("off")
plt.show()
def evaluate_accuracy_gpu(net, data_iter, device=None): #@save
    if isinstance(net, nn.Module):
        net.eval()  # 设置为评估模式
        if not device:
            device = next(iter(net.parameters())).device
    metric = d2l.Accumulator(2)
    with torch.no_grad():
        for X, y in data_iter:
            if isinstance(X, list):
                X = [x.to(device) for x in X]
            else:
                X = X.to(device)
            y = y.to(device)
            metric.add(d2l.accuracy(net(X), y), y.numel())
    return metric[0] / metric[1]
    #@save
def train_ch6(net, train_iter, test_iter, num_epochs, lr, device):
    def init_weights(m):
        if type(m) == nn.Linear or type(m) == nn.Conv2d:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    print('training on', device)
    net.to(device)
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    loss = nn.CrossEntropyLoss()
    animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs],
                            legend=['train loss', 'train acc', 'test acc'])
    timer, num_batches = d2l.Timer(), len(train_iter)
```

```python
    for epoch in range(num_epochs):
        metric = d2l.Accumulator(3)
        net.train()
        for i, (X, y) in enumerate(train_iter):
            timer.start()
            optimizer.zero_grad()
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            l.backward()
            optimizer.step()
            with torch.no_grad():
                metric.add(l * X.shape[0], d2l.accuracy(y_hat, y), X.shape[0])
            timer.stop()
            train_l = metric[0] / metric[2]
            train_acc = metric[1] / metric[2]
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (train_l, train_acc, None))
        test_acc = evaluate_accuracy_gpu(net, test_iter)
        animator.add(epoch + 1, (None, None, test_acc))
    print(f'loss {train_l:.3f}, train acc {train_acc:.3f}, '
          f'test acc {test_acc:.3f}')
    print(f'{metric[2] * num_epochs / timer.sum():.1f} examples /sec '
          f'on {str(device)}')
          lr, num_epochs = 0.9, 10
train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.try_gpu())
```

7 现代卷积神经网络

7.1 深度卷积神经网络 AlexNet

AlexNet 模型有比 LeNet 更大更深的学习网络，激活函数改用 Relu，同时池化方法采用的是最大池化。AlexNet 由 5 个卷积层，2 个池化层以及 3 个全连接层。以下是基于 Pytorch 实现的 AlexNet 模型示例代码：

```
    import torch
from torch import nn
from d2l import torch as d2l

net = nn.Sequential(
    # 这里使用一个11*11的更大窗口来捕捉对象。
    # 同时，步幅为4，以减少输出的高度和宽度。
    # 另外，输出通道的数目远大于LeNet
    nn.Conv2d(1, 96, kernel_size=11, stride=4, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(96, 256, kernel_size=5, padding=2), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(256, 384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 256, kernel_size=3, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Flatten(),
    # 这里，全连接层的输出数量是LeNet中的好几倍。使用dropout层来减轻过拟合
    nn.Linear(6400, 4096), nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(4096, 4096), nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(4096, 10))
X = torch.randn(1, 1, 224, 224)
```

```
for layer in net:
    X=layer(X)
    print(layer.__class__.__name__,'output shape:\t',X.shape)
batch_size = 128
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size,
    resize=224)
lr, num_epochs = 0.01, 10
d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.
   try_gpu())
```

7.2 使用块的网络 VGG

VGG 块：由一系列卷积层组成，后面加上用于空间下采样的最大汇聚层。

VGG 神经网络连接若干个 VGG 块。其中有超参数变量 conv-arch，该变量指定了每个 VGG 块里卷积层个数和输出通道数，全连接模块则与 AlexNet 中的相同。以下是基于 Pytorch 实现的 VGG 模型示例代码：

```
import torch
from torch import nn
from d2l import torch as d2l
def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels,
                                        kernel_size=3,
           padding=1))
        layers.append(nn.ReLU())
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2,stride=2))
    return nn.Sequential(*layers)
conv_arch = ((1, 64), (1, 128), (2, 256), (2, 512), (2, 512))

def vgg(conv_arch):
    conv_blks = []
```

```
    in_channels = 1
    # 卷积层部分
    for (num_convs, out_channels) in conv_arch:
        conv_blks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels

    return nn.Sequential(
        *conv_blks, nn.Flatten(),
        # 全连接层部分
        nn.Linear(out_channels * 7 * 7, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 10))
net = vgg(conv_arch)
X = torch.randn(size=(1, 1, 224, 224))
for blk in net:
    X = blk(X)
    print(blk.__class__.__name__,'output shape:\t',X.shape)
ratio = 4
small_conv_arch = [(pair[0], pair[1] // ratio) for pair in conv_arch]
net = vgg(small_conv_arch)
lr, num_epochs, batch_size = 0.05, 10, 128
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size, resize=224)
d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.try_gpu())
```

7.3 网络中的网络 NiN

NiN 块：由一个卷积层和两个 1×1 的卷积层，这两个卷积层用于合并通道，即当作全连接层使用。

NiN 由若干 NiN 块和若干最大池化层交替顺序分布，最后由全局平均汇聚层提供输出。以下是基于 Pytorch 实现的 NiN 模型示例代码：

```
import torch
from torch import nn
from d2l import torch as d2l
def nin_block(in_channels, out_channels, kernel_size, strides, padding):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size, strides, padding),
        nn.ReLU(),
        nn.Conv2d(out_channels, out_channels, kernel_size=1), nn.ReLU(),
        nn.Conv2d(out_channels, out_channels, kernel_size=1), nn.ReLU())
net = nn.Sequential(
    nin_block(1, 96, kernel_size=11, strides=4, padding=0),
    nn.MaxPool2d(3, stride=2),
    nin_block(96, 256, kernel_size=5, strides=1, padding=2),
    nn.MaxPool2d(3, stride=2),
    nin_block(256, 384, kernel_size=3, strides=1, padding=1),
    nn.MaxPool2d(3, stride=2),
    nn.Dropout(0.5),
    # 标签类别数是10
    nin_block(384, 10, kernel_size=3, strides=1, padding=1),
    nn.AdaptiveAvgPool2d((1, 1)),
    # 将四维的输出转成二维的输出，其形状为(批量大小,10)
    nn.Flatten())
X = torch.rand(size=(1, 1, 224, 224))
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__,'output shape:\t', X.shape)
    lr, num_epochs, batch_size = 0.1, 10, 128
```

```
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size, resize=224)
d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.try_gpu())
```

7.4 含并行连结的网络 GoogLeNet

Inception 块：将输入分别经过 4 个层通道，分别是 1×1 卷积层，1×1 卷积层和 3×3 卷积层，1×1 卷积层和 5×5 卷积层，3×3 最大汇聚层和 1×1 卷积层，最后这 4 个层通道经过通道合并层。

GoogLeNet 一共使用 9 个 Inception 块和全局平均汇聚层的堆叠来生成其估计值。Inception 块之间的最大汇聚层可降低维度。第一个模块类似于 AlexNet 和 LeNet，Inception 块的组合从 VGG 继承，全局平均汇聚层避免了在最后使用全连接层。以下是基于 Pytorch 实现的 GoogLeNet 模型示例代码：

```
import torch
from torch import nn
from torch.nn import functional as F
from d2l import torch as d2l

class Inception(nn.Module):
    # c1--c4是每条路径的输出通道数
    def __init__(self, in_channels, c1, c2, c3, c4, **kwargs):
        super(Inception, self).__init__(**kwargs)
        # 线路1, 单1x1卷积层
        self.p1_1 = nn.Conv2d(in_channels, c1, kernel_size=1)
        # 线路2, 1x1卷积层后接3x3卷积层
        self.p2_1 = nn.Conv2d(in_channels, c2[0], kernel_size=1)
        self.p2_2 = nn.Conv2d(c2[0], c2[1], kernel_size=3, padding=1)
        # 线路3, 1x1卷积层后接5x5卷积层
        self.p3_1 = nn.Conv2d(in_channels, c3[0], kernel_size
```

```
=1)
        self.p3_2 = nn.Conv2d(c3[0], c3[1], kernel_size=5, padding=2)
        # 线路4，3x3最大汇聚层后接1x1卷积层
        self.p4_1 = nn.MaxPool2d(kernel_size=3, stride=1, padding=1)
        self.p4_2 = nn.Conv2d(in_channels, c4, kernel_size=1)
    def forward(self, x):
        p1 = F.relu(self.p1_1(x))
        p2 = F.relu(self.p2_2(F.relu(self.p2_1(x))))
        p3 = F.relu(self.p3_2(F.relu(self.p3_1(x))))
        p4 = F.relu(self.p4_2(self.p4_1(x)))
        # 在通道维度上连结输出
        return torch.cat((p1, p2, p3, p4), dim=1)
b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
                   nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b2 = nn.Sequential(nn.Conv2d(64, 64, kernel_size=1),
                   nn.ReLU(),
                   nn.Conv2d(64, 192, kernel_size=3, padding=1),
                   nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b3 = nn.Sequential(Inception(192, 64, (96, 128), (16, 32), 32),
                   Inception(256, 128, (128, 192), (32, 96), 64),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b4 = nn.Sequential(Inception(480, 192, (96, 208), (16, 48), 64),
```

```
                    Inception(512, 160, (112, 224), (24, 64), 64),
                    Inception(512, 128, (128, 256), (24, 64), 64),
                    Inception(512, 112, (144, 288), (32, 64), 64),
                    Inception(528, 256, (160, 320), (32, 128), 128),
                    nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b5 = nn.Sequential(Inception(832, 256, (160, 320), (32, 128), 128),
                    Inception(832, 384, (192, 384), (48, 128), 128),
                    nn.AdaptiveAvgPool2d((1,1)),
                    nn.Flatten())

net = nn.Sequential(b1, b2, b3, b4, b5, nn.Linear(1024, 10))
X = torch.rand(size=(1, 1, 96, 96))
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__,'output shape:\t', X.shape)
lr, num_epochs, batch_size = 0.1, 10, 128
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size, resize=96)
d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.try_gpu())
```

7.5 批量归一化

批量归一化是指，对网络中间层的输出做标准化，将数据拉回到均值为 0、方差为 1 的分布，再通过可学习参数进行线性变换。其主要作用是缓解内部协变量偏移，使得数据允许使用更大学习率，加速训练收敛。

在深度学习框架中，可以使用已定义的 BatchNorm 方法实现。

7.6 残差网络 ResNet

通过捷径连接，让输入直接跳过部分卷积层，与层输出相加，学习残差映射而非原始映射。

残差网络可以解决深度网络退化问题，让网络可以做得很深，同时让梯度可通过捷径直达浅层，缓解梯度消失。

ResNet 模型的前两层跟之前介绍的 GoogLeNet 中的一样：在输出通道数为 64、步幅为 2 的 7×7 卷积层后，接步幅为 2 的 3×3 的最大汇聚层。不同之处在于 ResNet 每个卷积层后增加了批量归一化层。

以下是基于 Pytorch 实现的 ResNet 模型示例代码：

```
import torch
from torch import nn
from torch.nn import functional as F
from d2l import torch as d2l


class Residual(nn.Module):  #@save
    def __init__(self, input_channels, num_channels,
                 use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.Conv2d(input_channels, num_channels,
                               kernel_size=3, padding=1, stride
                                  =strides)
        self.conv2 = nn.Conv2d(num_channels, num_channels,
                               kernel_size=3, padding=1)
        if use_1x1conv:
            self.conv3 = nn.Conv2d(input_channels, num_channels
                ,
                                   kernel_size=1, stride=
                                      strides)
        else:
```

```
            self.conv3 = None
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.bn2 = nn.BatchNorm2d(num_channels)

    def forward(self, X):
        Y = F.relu(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return F.relu(Y)
blk = Residual(3,3)
X = torch.rand(4, 3, 6, 6)
Y = blk(X)
Y.shape
blk = Residual(3,6, use_1x1conv=True, strides=2)
blk(X).shape
b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2,
    padding=3),
                   nn.BatchNorm2d(64), nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2,
                       padding=1))
def resnet_block(input_channels, num_channels, num_residuals,
                 first_block=False):
    blk = []
    for i in range(num_residuals):
        if i == 0 and not first_block:
            blk.append(Residual(input_channels, num_channels,
                                use_1x1conv=True, strides=2))
        else:
            blk.append(Residual(num_channels, num_channels))
    return blk
b2 = nn.Sequential(*resnet_block(64, 64, 2, first_block=True))
```

```
b3 = nn.Sequential(*resnet_block(64, 128, 2))
b4 = nn.Sequential(*resnet_block(128, 256, 2))
b5 = nn.Sequential(*resnet_block(256, 512, 2))
net = nn.Sequential(b1, b2, b3, b4, b5,
                    nn.AdaptiveAvgPool2d((1,1)),
                    nn.Flatten(), nn.Linear(512, 10))
X = torch.rand(size=(1, 1, 224, 224))
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__,'output shape:\t', X.shape)
lr, num_epochs, batch_size = 0.05, 10, 256
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size,
    resize=96)
d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.
   try_gpu())
```

8 注意力机制

8.1 注意力提示

引入注意力显式地考虑随意线索，随意线索称之为查询，每个输入是一个键值对，通过注意力池化层选择某些输入。

8.2 注意力汇聚：Nadaraya-Watson 核回归

注意力汇聚是指，根据查询（待预测点）与键（训练点）之间的距离，用核函数计算注意力权重，对值（标签）做加权平均，得到预测输出。离查询越近的训练样本，权重越大，对预测贡献越强。

8.3 注意力分数

在注意力机制中，用于衡量 Query 与 Key 之间相似度的原始得分，注意力分数暗示着数据重要程度。

8.4 自注意力

Query、Key、Value 都来自同一组输入序列的注意力机制，自己和自己做注意力匹配。看序列里每个位置，和序列里所有位置的相关性，自己决定该关注自己序列里的谁。

8.5 多头注意力

多头注意力是 Transformer 架构的核心，通过并行计算多个独立的注意力头，从不同子空间捕捉向量序列的不同关系。以下是基于 Pytorch 实现的多头注意力模型示例代码：

```
import math
import torch
from torch import nn
from d2l import torch as d2l
#@save
class MultiHeadAttention(nn.Module):
    def __init__(self, key_size, query_size, value_size,
        num_hiddens,num_heads, dropout, bias=False, **kwargs):
        super(MultiHeadAttention, self).__init__(**kwargs)
        self.num_heads = num_heads
        self.attention = d2l.DotProductAttention(dropout)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=bias
            )
        self.W_k = nn.Linear(key_size, num_hiddens, bias=bias)
        self.W_v = nn.Linear(value_size, num_hiddens, bias=bias
            )
        self.W_o = nn.Linear(num_hiddens, num_hiddens, bias=
            bias)

    def forward(self, queries, keys, values, valid_lens):
        queries = transpose_qkv(self.W_q(queries), self.
            num_heads)
        keys = transpose_qkv(self.W_k(keys), self.num_heads)
```

```
        values = transpose_qkv(self.W_v(values), self.num_heads)

        if valid_lens is not None:
            valid_lens = torch.repeat_interleave(
                valid_lens, repeats=self.num_heads, dim=0)
        output = self.attention(queries, keys, values, valid_lens)
        output_concat = transpose_output(output, self.num_heads)
        return self.W_o(output_concat)
def transpose_qkv(X, num_heads):
    X = X.reshape(X.shape[0], X.shape[1], num_heads, -1)
    X = X.permute(0, 2, 1, 3)
    return X.reshape(-1, X.shape[2], X.shape[3])
def transpose_output(X, num_heads):
    X = X.reshape(-1, num_heads, X.shape[1], X.shape[2])
    X = X.permute(0, 2, 1, 3)
    return X.reshape(X.shape[0], X.shape[1], -1)
num_hiddens, num_heads = 100, 5
attention = MultiHeadAttention(num_hiddens, num_hiddens, num_hiddens,num_hiddens, num_heads, 0.5)
attention.eval()
batch_size, num_queries = 2, 4
num_kvpairs, valid_lens =  6, torch.tensor([3, 2])
X = torch.ones((batch_size, num_queries, num_hiddens))
Y = torch.ones((batch_size, num_kvpairs, num_hiddens))
attention(X, Y, Y, valid_lens).shape
```

8.6 transformer 架构

transformer 架构是编码器-解码器的一个实例。其中编码器每一层包含一个多头自注意力层，两个残差链接归一化层，和一个前馈网络层；解码器包含三个

模块：掩蔽多头注意力层，多头注意力层，逐位前馈网络层。在解码器自注意力中，查询、键和值都来自上一个解码器层的输出。但是，解码器中的每个位置只能考虑该位置之前的所有位置。